

Prague University of Economics and Business

Faculty of Informatics and Statistics



HIGH-UTILITY ACTION RULES MINING

BACHELOR THESIS

Study programme: Applied informatics

Field of study: Applied informatics

Author: Kateřina Hrudková

Supervisor: Ing. Lukáš Sýkora

Prague, May 2022

Acknowledgement

I hereby wish to express my appreciation and gratitude to the supervisor of my thesis, Ing. Lukáš Sýkora.

Abstract

High utility action rules mining is a data mining task, that aims to generate such rules, that would provide the user with the information of which actions might be the most profitable. To the author's best knowledge, no open-source solution for high utility action rules mining has not been proposed to date.

The aim of this paper is to present reader with brief background of the key topics (frequent itemset mining, action rules mining, high utility itemset mining) related to the high utility action rules mining and ActionRules package and to implement a practical solution in Python, to the existing ActionRules package framework, that would enable high utility action rules mining.

The chapters on frequent itemset mining, action rules mining, high utility itemset mining and part of the last chapter on high action rules mining were processed by the method of narrative literature search. Through the method of analysis, the workflow of ActionRules package was observed and on the insights of the analysis the aimed high utility action rules mining solution has been implemented to the existing ActionRules framework.

Keywords

action rules, high utility action rules mining, high utility itemset mining, Python.

JEL Classification

C88

Abstrakt

Dolování akčních pravidel s vysokou hodnotou užitku je dataminingová úloha, která uživateli poskytuje informaci o tom, jaké akce by mohly přinést nejvyšší užitek. Dle nejlepšího vědomí autora, zatím neexistuje open-source implementace pro dolování akčních pravidel s vysokou hodnotou užitku.

Cílem této práce je představit čtenáři stručný souhrn klíčových témat (dolování častých itemsetů, dolování akčních pravidel, dolování itemsetů s vysokou hodnotou užitku) souvisejících s dolováním akčních pravidel s vysokou hodnotou užitku a knihovnou ActionRules, a implementovat praktické řešení v Pythonu k existující knihovně ActionRules, které by umožnilo dolování akčních pravidel s vysokou hodnotou užitku.

Kapitoly týkající se dolování častých itemsetů, dolování akčních pravidel, dolování itemsetů s vysokým užitekem a část poslední kapitoly na dolování akčních pravidel s vysokým užitekem byly zpracovány metodou narativní literární rešerše. Metodou analýzy byl pozorován workflow knihovny ActionRules a na základě postřehů z analýzy, cílené řešení pro dolování akčních pravidel s vysokou hodnotou užitku bylo implementováno do existující knihovny ActionRules.

Klíčová slova

akční pravidla, dolování akčních pravidel s vysokou hodnotou užitku, dolování itemsetů s vysokou hodnotou užitku, Python.

JEL Klasifikace

C88

Content

Introduction	6
1 Frequent itemset mining.....	8
1.1 Key applications of FIM.....	9
1.2 Frequent itemset mining algorithms	11
1.2.1 Apriori algorithm	12
1.2.2 Apriori-C.....	13
2 Action rules mining	16
2.1 Action rules mining algorithms.....	17
2.1.1 Loosely coupled framework	17
2.1.2 Tightly coupled framework.....	18
3 High utility itemset mining	20
3.1 High utility itemset mining algorithms.....	21
3.1.1 Two phase algorithms	22
3.1.2 Pattern-Growth Two Phase Algorithms	24
3.1.3 One phase algorithms	25
3.1.4 Patter-Growth One-Phase Algorithms	26
4 High utility action rules mining	27
4.1 Package ActionRules	27
4.2 Extension for high utility action rules mining.....	28
4.2.1 Flow diagram of the extended ActionRules package.....	29
4.2.2 Utility source	30
4.2.3 Pruning candidate pairs by difference in utility.....	32
4.2.4 Sorting output action rules by utility of action	33
4.2.5 Output rules interpretation.....	33
4.3 Alternative implementations	34
4.4 Future development.....	34
Conclusions	36
List of references	37
Annexes	I
Annex A: Jupyter notebook Employees.....	I
Annex B: Source dataset Employee Attrition	I
Annex C: Utility table for Employee Attrition dataset.....	I

Introduction

For the past several decades, data scientists have been discovering all kind of information from the data, using various knowledge discovery methods. In the world, where data logger devices are very common and the volumes of data increase every minute, the processing of data and discovering insightful information from it is more and more important.

One of the knowledge discovery methods is data mining. Data mining methods aim to discover patterns in database systems. These patterns, depending on the nature of the pattern and the method used for its discovery, present user with an information. Probably the most broadly studied is frequent itemset mining – a data mining method focusing on discovering patterns frequently appearing in databases. But is information solely about the frequency of the pattern always sufficient, to derive insightful conclusions, when, for example, examining a situation, where employees are leaving the company in bigger numbers?

Frequent itemset mining, though broadly used and studied, usually require some sort of post-processing, to get useful answers. Throughout the time, it has been more used as an intermediate step in other data mining methods – association rules, classification, clustering, just to name few. One of the modifications of the frequent itemset methods is high utility itemset mining – a data mining method, that discovers the most profitable patterns.

Given the problem of the employees leaving the company, frequent itemset mining and its application of association rules mining can give an information about the characteristics of the employees that left the company. However, company would most likely want to prevent such situation from happening – even though the association rules can give some characteristics, it cannot always be clear, which are the ones, that push the employee over the edge. For that, action rules can be used, as they suggest, what kind of characteristic can be changed to make a difference, whether the employee stays or leaves.

However, such changes, if they are possible, can be costly and might be cause even more harm when implemented, then if they were not. By combining the aspects of action rules and high utility itemsets, it is possible to generate rules, that only suggest changes, that user finds profitable.

This paper aims to implement an open-source solution for high utility action rules mining as well as provide brief background of information about related topics and existing algorithms, to help reader understand the key concepts, the idea of high utility action rules mining and the existing and extended ActionRules package implementation.

The part of this paper focusing on the information background of key topics will be processed by method of narrative literature search of related sources. The understanding of the existing framework will be processed using the method of analysis. Using the insights

from the analysis the practical solution for high utility action rules mining will be implemented to the existing ActionRules package.

Frequent itemset mining is described in chapter 1 of this paper, action rules mining in chapter 2 and high utility itemset mining in chapter 3. The Python implementation of the high utility action rules mining is presented and demonstrated in chapter 4.

1 Frequent itemset mining

Frequent itemset mining (FIM, also called frequent pattern mining) is one of the best-known data mining methods. Its aim is to find regularities (patterns) between variables in a particular data set (Borgelt, 2012). It is usually related to descriptive task that aims at finding patterns that denote homogenous and regular behaviour on unlabelled data (Luna et al., 2019).

$I = \{i_1, i_2, \dots, i_m\}$ is a set of items (also called item base), $\{i_1, i_2, \dots, i_n\} \subseteq I$ is an itemset.

Transaction database D is a set of transactions $D = \{T_1, T_2, \dots, T_n\}$ where each transaction $T \in D$ is a subset of I .

Table 1.1 Transaction database

Tid	Transaction
1	{a}
2	{a, b, c}
3	{b, c}
4	{a, c, d}
5	{b, d}

Depending on the dataset, the number of discovered frequent itemsets by FIM could be overwhelmingly large. It is not unusual, that the size of the output may be larger than the size of the input (Aggarwal, 2014). Generally, FIM is considered computationally challenging and during the first few years of research in area of frequent pattern mining the primary focus of work was to find algorithms with better computational efficiency (Aggarwal et al., 2014) though it did not take long and researches became aware of the importance to not only propose efficient algorithmic solutions but also new approaches – such as parallel and distributed computing – to handle the computational complexity (Luna et al., 2019).

“The FIM task is a really tough problem due to the large number of candidate patterns (itemsets) that are required to be handled. Considering a dataset comprising n items the number of patterns (or itemsets) of size k is $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ for any $k \leq n$. As a result, the total number of feasible patterns is $2^n - 1$ for such dataset, and the complexity of finding patterns of interest is in exponential order. This complexity is even higher when the frequency of each pattern is calculated, resulting as $O((2^n - 1) \times m \times n)$ for a dataset

comprising m records and n items. Aiming at shedding some light on the complexity, a dataset comprising just 33 items (a low number) produces more than 8.5×10^9 feasible patterns.” (Luna et al., 2019)

The search space is filtered through the anti-monotone property of support or in other words – frequency (Luna et al., 2019). Anti-monotone property states, that all non-empty subsets of frequent itemsets must also be frequent – the same applies for all non-empty subsets of infrequent itemsets. This is also known as Apriori property or downward closure property (Aggarwal et al., 2014).

Support is a relative number of transactions covering the given itemset (Agrawal et al., 1993).

1.1 Key applications of FIM

The range of applications of FIM is wide and spans across variety of fields and data domains. Frequent patterns are often difficult to use directly without using some form of post-processing. In practice, outputs of FIM procedures are often used as intermediary steps in other data mining applications (Aggarwal, 2014).

The very first application of FIM was proposed in the context of customer analysis where customer behaviour is depicted by set of items bought together. In this context, the frequent patterns are used to determine customer’s buying behaviour. Such information can be further used for product placement or making promotion decisions (Aggarwal, 2014).

Frequent itemsets are also used to find associations in data, called association rules, first proposed and applied in 1993 by Agrawal et al. on data from retailing company. The association rules are implications and have form of:

$$\text{Antecedent } X \Rightarrow \text{Consequent } Y$$

where X and Y are sets of items $X, Y \subseteq I, X \cap Y = \emptyset$ and $X, Y \neq \emptyset$. The left-hand side of the rule is also called the antecedent of the rule, the right-hand side is called the consequent of the rule.

In other words, the association rule implies that transaction that contains items in X tend to also contain items in Y (Strikant & Agrawal, 1997).

The association rule mining consists of two subproblems – discovering frequent itemsets and using these itemsets to generate the rules. There are two basic measures for association rules – support of the rule, concerning the number of transactions supporting the rule and confidence, described as measure of rule’s strength.

“The confidence of an association rule is a percentage value that shows how frequently the rule head occurs among all the groups containing the rule body. The confidence value indicates how reliable this rule is. The higher the value, the more likely the head items occur in a group if it is known that all body items are contained in that group.

Thus, the confidence of a rule is the percentage equivalent of m/n , where the values are:

- m = the number of groups containing the joined rule head and rule body¹
- n = the number of groups containing the rule body”

(IBM, 2021)

In terms of association rules mining other than constraints on support and confidence Agrawal et al. (1993) notes also syntactic constraints, which are described as restrictions on items that can appear in a rule. It is possible to put such restriction to filter only these rules, that only have specific range of items in the antecedent or consequent. This restriction can also be used to generate rules in a form, that allows them to be used for classification. Therefore, classification rule mining is another one of the applications closely related to FIM (and key application for action rules mining). Thus, through broadly used association mining methods it is also possible to find patterns sufficiently discriminative for classification (Aggarwal, 2014). The integrated framework of association and classification rule mining is called associative classification (Bing et al., 1998) (more on rule-based classification in section 1.2.2).

In its role as an intermediate step in other data mining procedures, FIM is closely related to outlier detection and clustering (and its use for example in the domain of biological data – biclustering (Madeira & Oliveira, 2004)). Transaction clustering, which refers to partitioning a collection of transactions into clusters, is based on, rather than adopting pairwise similarity approach for measuring distance, notion of frequent items (Wang et al., 1999). *“An item is large² in a cluster of transactions if it is contained in a user-specified fraction of transactions in that cluster. This user-specified fraction is called the minimum support. We use large items to measure the similarity of the cluster. Our criterion of a good clustering is that there are many large items within a cluster and there is little overlapping of such items across clusters.”* (Wang et al., 1999)

Some of the other key applications of FIM that Aggarwal (2014) states are frequent patterns for indexing, web mining applications and use of frequent patterns for text mining. Temporal applications, spatial and spatiotemporal applications, software bug detection, chemical and biological applications and resources for the practitioner are also mentioned to be applications of FIM in the same paper.

Modification of FIM problem called high utility itemset mining is further discussed in chapter 3.

¹ The rule body in this definition refers to the antecedent of the rule, the rule head refers to the consequent of the association rule.

² Authors in this citation use term “large” items. It comes from the fact, that frequent itemset mining was initially called large itemset mining back in early nineties (Luna et al., 2019)

1.2 Frequent itemset mining algorithms

In the earlier years of the FIM algorithms development any solution proposed followed level-wise breadth first search based that produced candidate itemsets which were validated against transaction database in each iteration of the algorithm (Luna et al., 2019). The first and base algorithm for all the early FIM algorithms is called Apriori and is further discussed in the following section of this chapter.

The first algorithmic breakthrough in the task of FIM happened in 2000 when the algorithm FP-growth was proposed. The main arguments for developing new approach for frequent itemset mining were, that it is costly to handle huge candidate sets and that its tedious to repeatedly scan database to check a large set of candidates by pattern matching. For the purpose of overcoming these weaknesses of the Apriori approach, the FP-growth algorithm introduces frequent itemset mining through the novel data structure called frequent pattern tree (FP-tree), which is an extended prefix-tree structure storing information about frequent patterns. The growth of patterns is achieved via concatenation of the suffix pattern with the ones generated from the FP-tree. The number of the database scan is reduced to exactly two – first collects the set of frequent items and the second creates the FP-tree (Han et al., 2000).

FP-tree is a tree structure consisting of

- one root labelled as “null”, a set of item prefix subtrees as the children of the root and a frequent item header table
- nodes of the item prefix subtree all consist of item-name, count (the number of transactions represented by portion of the part reaching this node) and node-link
- each entry in the frequent-item header table consists of two files – item-name and head of node-link which points to the first node in the FP-tree carrying the item-name

(Han et al., 2000)

FP-growth proved to be way faster than Apriori and to this day is a key algorithm to compare with (Luna et al., 2019).

Around the same time in 2001 the ECLAT algorithm was proposed which associates to each pattern the list of transactions in which the pattern occurs, therefore the frequency can be obtained by intersecting the (Zaki, 2000; Luna et al., 2019). Similar algorithm dECLAT dealing with the problem of intersecting large number of transaction list was three years later proposed, that keeps track of diffsets and frequent itemsets are generated by computing diffsets for all distinct pairs of itemsets and checking the support of result (Zaki & Gouda, 2003).

Diffset is vertical data representation that keeps track of differences in the *tids* of candidate pattern from its generating patterns (Zaki & Gouda, 2003).

Among other well-known FIM algorithms is the winner of FIMI'04 (Frequent Itemset Mining Implementations) competition, the LCM algorithm, which pushed the research in direction of non-exhaustive approach rather than traditional exhaustive one, PFP, based on MapReduce programming framework, which worked on distributed machines and its variations (SPC, FPC, DPC) and even better MapReduce algorithms proposed in 2008 DistECLAT and BigFIM (Luna et al., 2019).

1.2.1 Apriori algorithm

Apriori algorithm is one of the best-known and earliest algorithms for frequent itemset mining over transaction database. It was proposed by Rakesh Agrawal and Ramakrishnan Strikant in 1994 and was designed to work on a set of binary attributes (items) and transaction database where each transaction was represented as vector of binary attributes (itemsets) (Luna et al., 2019). The original motivation for searching such itemsets came from the need to analyze data about customer's purchased products – and frequent sets of purchased products can give information, about how often are items purchased together. The most popular practical example describing Apriori algorithm is called Market Basket Analysis. It is worth to mention, as hinted in the section 1.2 on FIM algorithms, that till year 2000, any FIM algorithm proposed was just a small variation of Apriori (Luna et al., 2019).

As name of the algorithm suggests, the algorithm uses prior knowledge about the properties of the frequent itemset. Apriori applies breadth-first search and is a join-based algorithm – join-based algorithms generate $(k + 1)$ -candidate itemsets from known frequent k -patterns with the use of joins and these candidates are validated against the transaction database. Candidates are validated against the transaction database by calculating support. The approach of generating itemsets of length k is called level-wise approach. It is possible to produce the same candidate itemset by joining multiple frequent patterns. This duplication of patterns is avoided by joining only such pair of itemsets that have the same first $(k - 1)$ items based on lexicographic ordering imposed on the items (Aggarwal et al., 2014).

A join is defined by pairs of frequent k -patterns that have at least $(k - 1)$ items in common (Aggarwal et al., 2014).

Apriori algorithm consists of three steps which run over and over. The process of finding frequent itemset starts with deriving candidate itemsets in transaction database, which are considered frequent. These candidate itemsets are being extended by another single item. Once the support measure of any of these itemsets drops below the minimal threshold, they are not being further extended (Agrawal et al., 1993). Pruning candidates without validating against transaction database is also possible – taking the anti-monotone property into account, if all k subsets of $(k + 1)$ -candidate are infrequent, the candidate can be pruned from consideration. This is known as Apriori pruning trick, which greatly speeds up the overall algorithm (Aggarwal et al., 2014). The process is terminated once the returned frequent itemset is empty.

The setback of the Apriori algorithm is that it is slow and its generally not known for a great performance. Database is scanned every time new item is being searched and support is counted for every new itemset, which can take very long, depending on the size of the

database. The complexity of Apriori is $O(m^2n)$, where m represents the number of distinct items and n the number of transactions (Fournier-Viger et al., 2019).

Optimalizations for Apriori algorithm were proposed in 1994 called AprioriTid and AprioriHybrid (Agrawal & Strikant, 1994).

AprioriTid algorithm does not use transaction database for counting support – instead a new set, consisting of a pair $tid, \{L_k\}$, is used. That means each transaction is replaced by a shorter or empty transaction during the k th phase. The transaction identifiers in the set are matched to the corresponding identifiers in transaction database – if a transaction in database does not contain any candidate itemset it will not have an entry in the set (Agrawal & Strikant, 1994; Luna et al., 2019). Aggarwal (2014) in his survey on FIM algorithms mentions, that this optimization should not be used in the early iterations of the algorithm and should only be applied in the later iterations, because more than one candidate may be subset of the transaction, which will increase the overhead of the algorithm – this variation is called AprioriHybrid.

The ActionRules package uses modified Apriori algorithm called Apriori-C algorithm, which can be used for classification and is further discussed in the section 1.2.2 of this chapter.

Pseudo-code

Code 1.1 Apriori algorithm pseudo-code (Agrawal & Srikant, 1994)

```

 $L_1 = \{\text{frequent 1-itemsets}\};$ 

for ( $k = 2; L_{k-1} \neq 0; k++$ ) do begin
     $C_k = \text{apriori-gen}(L_{k-1});$ 
    for each transaction  $t \in D$  do begin
         $C_t = \text{subset}(C_k, t);$ 
        for each candidate  $c \in C_t$  do
             $c.\text{count}++;$ 
        end
         $L_k = \{c \in C_k \mid c.\text{count} \geq \text{minsup}\}$ 
    end
end
return  $\bigcup_k L_k$ 

```

1.2.2 Apriori-C

The ActionRules package implements a rule-based approach, in which action rule is generated from pair of classification rules. Apriori-C is a classification rule learning algorithm proposed in 2004 by Jovanoski & Lavrač. Classification is about classifying data into categories, where they share some common features. The aim of classification rule mining is to find set of rules that forms an accurate classifier (Bing et al., 1998). Classifier is usually presented in the form of list of the rules (Coenen & Leng, 2004).

Classification rules are subset of association rules. Rule based classification uses set of IF-THEN rules for classifying data (also called decision rules). The IF part of the rule – antecedent – represents a condition the THEN part – consequent – represents predicted class. Antecedent consists of one or more attributes; consequent consists of a single target attribute. That is the difference between the association rules mining and classification rules mining – classification rule has one and only one pre-determined target attribute in the consequent – the target class (Liu et al., 1998).

Several mechanisms for using such classifier exist – Best Rule mechanism selecting the first rule that satisfies the given case according to an ordering imposed on the rule listing (CSA ordering, WRA, Laplace accuracy, X^2 Testing etc.), Best K rules mechanism selecting the first best K rules that satisfy the given case and then select a rule according to some averaging process (satisfaction of the case also according to imposed ordering in the rule listing) and All rules mechanism collecting all rules of the classifier that satisfy the given case and then evaluate this collection to identify a class (Wenmin et al., 2001).

The modified Apriori-C algorithm is capable to generate classification rules by saving only the supported itemsets of sizes k and $k + 1$. That is because only the rules having the target attribute in the consequent are need (Jovanoski & Lavrač, 2001). These itemsets are otherwise generated the same way as with the vanilla Apriori algorithm.

Unneeded rules (rules for which different rules exist and are their generalization) are pruned by exclusion of corresponding itemsets from the set of $k + 1$ -candidates. Itemsets containing unneeded items, that cannot be found in any itemset together with the target item, are also pruned from the search (Jovanoski & Lavrač, 2001).

Apart from Apriori-C algorithm, other known algorithms for generating rules to build classifiers are for example CBA-RG algorithm, the algorithm proposed back in 1998 by Bing et al. along with the concept of associative classification, CAEP algorithm, proposed year later in 1999 by Dong et al., built on emerging patterns (proposed by Dong & Li, 1999), which captures multi-attribute contrasts between data classes and trends over time across different datasets, ADT algorithm (Wang et al., 2000) mining rules only based on confidence or CMAR, associative classification method based on multiple association rules proposed in 2001 by Wenmin et al.

Pseudo-code

Code 1.2 Apriori-C algorithm pseudo-code (Jovanoski & Lavrač, 2001)

```
for each target item  $T$ 
     $k = 1$ 
     $C_1$  = set of all 1-itemsets
    check the support of all itemsets in  $C_1$ 
    delete unsupported itemsets from  $C_1$ 
    while  $C_k$  not empty do
        build all potentially supported  $k + 1$  itemsets
        put them into  $C_{k+1}$ 
        check their actual support
        delete unsupported itemsets from  $C_{k+1}$ 
        find all rules that have target item  $T$  at their right-hand side
        delete all itemsets that were used for constructing rules
        detect unneeded items and delete unneeded itemsets
         $k = k + 1$ 
    end while
endfor
```

2 Action rules mining

As it has been previously mentioned in the chapter on FIM, finding frequent patterns in data is one of the best-known data mining methods. These methods focus on generating huge numbers of patterns in data. However, to get useful knowledge from data, the patterns need to be further analyzed. The action rule mining is a data mining method, where the output (action rules) is going beyond description of a pattern.

The concept of action rules mining was first introduced in 2000 by Zbigniew Wiesław Raś and Alicja Wierczorkowska to answer specific questions about customers of a bank company – for example question about actions improving the profitability of customers. The authors put emphasis on importance of understanding the customer as well as keeping a broad perspective on knowledge. The proposed algorithm in the same paper is used to identify customers which may move from one profit ranking group to another based on change in some of their characteristics. And that is the objective of action rules mining – the purpose of action rule is that it suggests a change in attributes in the antecedent of the rule that will consequently change (re-classify) the target attribute.

There are two known frameworks for generating action rules – loosely coupled and tightly coupled. The loosely coupled – often called rule-based – is based on generating action rules from classification rules. This approach can be further subdivided to methods either using pairs of classification rules to generate an action rule or methods generating action rules from single classification rules. The tightly coupled – often called object-based – assumes that action rules can be discovered directly from a database (Dardzinska, 2013). As was previously mentioned in the section on Apriori-C, the ActionRules package uses rule-based approach that pairs classification rules to generate action rules therefore one of the important steps is classification rules mining.

*“Expression $r1 = [((a, a2) * (b, b2 \rightarrow b1)) \rightarrow (d, H \rightarrow A)]$ is an example of action rule. The rule $r1$ says that if the value $a2$ remains unchanged and value b will change from $b2$ to $b1$ for a given object x , then it is expected that the value d will change from H to A for object x .” (Dardzinska, 2013)*

Action rules can be seen as logical terms describing knowledge about possible actions. Classification rule consists of antecedent and consequent and is used to display classification of data. Action rule is supposed to take that and suggest an action in the antecedent, that would change the value of the consequent – the result of classification would change. Therefore, the action rules are understood as the analysis and identification of operations that should be taken to get the desired state (Sikora et al., 2021).

Loosely coupled framework algorithms mine action rules from information system called decision table. A decision table is an information system of the following form:

$$S = (U, A_1 \cup A_2 \cup \{d\}) \text{ (Raś \& Wierczorkowska, 2000)}$$

where U is a nonempty, finite set called the universe, A is a nonempty, finite set of attributes (Pawlak, 1985), A_1 is a set of stable attributes, which values always remain unchanged, A_2 is a set of flexible attributes, which values can be changed so that the desired state is reached. d , for which applies $d \notin A_1 \cup A_2$, represents the target attribute, also called decision (Raś & Wierczorkowska, 2000). Most of the action rules mining algorithms are based on the Pawlak's definition of an information system.

Decision table serves as an input to a LERS system (algorithms LEM1, which finds the set of all coverings and introduces marking system and LEM2, which finds certain and possible rules for a given value of a decision attribute). LERS system induces a set of classification rules from examples given in the form of decision table and may classify new examples – it belongs to the class of machine learning systems from examples. The system searches for regularities in input data and is capable of handling uncertainty – missing values and inconsistent examples (Chmielewski et al., 1993). On similar principle functions algorithm ERID, used for extracting rules from incomplete decision systems. These systems for rule induction and marking of subsets of the decision attributes serves as a core for some of the action rules mining systems.

Tightly coupled framework algorithms do not use decision table as input.

2.1 Action rules mining algorithms

2.1.1 Loosely coupled framework

Loosely coupled framework for action rules mining generates action rules from classification rules. While generating actions rules classification rules the threshold on support and confidence measures is still in place – when generating from a pair of classification rules, both of the classification rules have to have all measures at least equal to a minimal threshold to be considered for action rules mining.

One of the algorithms Dardzinska (2013) mentions is called system DEAR, which generates action rules from pairs of classification rules. System DEAR identifies objects in information system that can be moved to another group of objects, if the values of some flexible attributes describing them changes. It also identifies the optimal set of attributes which values should be changed and their new values (Dardzinska, 2013). System DEAR introduces the concept of d -reduct.

“The concept of d -reduct in S was introduced to induce rules from S describing values of the attribute d depending on minimal subsets of $A_1 \cup A_2$ which preserve the confidence of extracted rules. In order to induce rules in which the then part consists of the decision attribute d and the if part consists of attributes belonging to $A_1 \cup A_2$, sub-tables $(X, B \cup \{d\})$ of S where B is a d -reduct in S should be used for rules extraction.” (Dardzinska, 2013)

System DEAR2 is an implementation of an algorithm called Action Tree. The algorithm searches for a solution in a part of the search space – it seeks at each stage for a stable attribute that has a least number of values. The set of rules is then split recursively using

that attribute. Once all stable attributes are processed, the final subsets are further split based on a decision attribute – this generates an action tree. This algorithm consists of two main steps – building the action tree and generating action rules by comparing all unmarked leaf nodes of the same parent (Dardzinska, 2013).

System ARAS implements one of the rule-based algorithms, that use a single classification rule for action rule building, rather than the pairs. Raś et al. (2008) states that constructing action rules from pairs of classification rules is not only unnecessarily expensive, but it also gives too much freedom in the construction of the classification rules – therefore, propose system ARAS in the same paper, a LERS-type algorithm for constructing action rules from single classification rule. Using LERS as pre-processing module for ARAS decreasing the overall complexity of the algorithm because LERS do the validity checks on the input rules. As opposed to DEAR system, which groups classification rules into clusters of non-conflicting rules and then tries to build action rules from all possible pairs of classification rules within cluster, ARAS treats each classification rule describing the desired target value as a seed and forms the cluster with other classification rules and then it builds decision rules automatically from them – rules grabbed into a seed are only compared with that seed, reducing the number of pairs that have to be checked in comparison to DEAR. For comparison, the complexity of action rules generation phase of these two systems is $O(k \times n)$ for ARAS, where n is the number of classification rules and k is the number of clusters and $O(n \times n)$ for DEAR, where n is the same as in ARAS (Raś et al., 2008).

2.1.2 Tightly coupled framework

Tightly coupled framework presents methods for action rules mining without using pre-existing classification rules. The first algorithm attempting to achieve this was an Apriori-like algorithm called MARFS1 proposed by He et al. in 2005 stating the action rules mining as a search problem in a support-confidence-cost framework. This framework allows stable attributes to be subjected to change as well as the flexible ones, however these changes are to be pruned out according to a cost of an action rule – changes of stable attribute are set to very large numbers, so that these rules are not considered. The cost is also used to prune out such changes, that are too costly (determined by domain experts) to be useful (He et al., 2005). MARFS1 algorithm operates based on downward-closure property on support and cost to find frequent action rules – confidence is not considered in candidate generation phase.

This way of discovering action rules as however deemed unnecessarily costly – a new strategy was proposed by Raś & Dardzinska in 2008 – ARD algorithm – which uses atomic expressions to build action rules, authors compare its similarity in marking and combining the expressions to system ERID.

“Atomic action term is an expression($a, a_1 \rightarrow a_2$), where a is an attribute, if $a_1 = a_2$ then a is called stable on a_1 .” (Dardzinska, 2013)

*“If t is an atomic action term, then t is an action term, if t_1, t_2 are action terms, then $t_1 * t_2$ is an action term.” (Dardzinska, 2013)*

“An action rule is an expression $r = [t_1 \rightarrow t_2]$, where t_1 is an action term, and t_2 is an atomic action term.” (Dardzinska, 2013)

3 High utility itemset mining

High utility itemset mining (HUIM, also called high utility pattern mining) is a data science task which focuses on discovering high utility itemsets in quantitative transaction databases (Fournier-Viger et al., 2019).

High utility pattern mining was defined to address limitations of frequent itemset mining. Frequent itemsets provide information about frequency of the presence or absence of an item in transaction database. But just because itemset is frequent it does not mean that it significantly contributes to the overall profit. Utility mining is generalization of frequent itemset mining – items in the database are annotated with numerical values and itemsets are selected based on a user-defined utility function rather than just frequency. Therefore, the goal of utility mining is to discover all the itemsets, which utility is above user specified threshold (Liu et al., 2005).

Itemset I is high utility itemset if its utility $u(I)$ is no less than minimum utility threshold $minutil$ specified by user ($u(I) \geq minutil$) – otherwise I is a low utility itemset (Fournier-Viger et al., 2019).

A quantitative transaction database is a transaction database, where each item is associated with a positive number – this number is the external utility. The utility of an item or itemset is based on external utility and internal utility. The external utility of an item is a positive number represents relative importance to its user. This number is based on information from resources like profit table rather than just transactions alone. The internal utility of an item is a positive number which represents quantity of that item in a transaction (Fournier-Viger et al., 2019).

Table 3.1 Quantitative transaction database

Tid	Transaction
1	{(a, 3)}
2	{(a, 5), (b, 2), (c, 3)}
3	{(b, 2), (c, 3)}
4	{(a, 2), (c, 6), (d, 4)}
5	{(b, 1), (d, 3)}

Table 3.2 Table of external utilities

Item	External utility
a	5
b	2
c	3
d	4

The utility of an item in transaction is a multiplication of its internal and external utility. In case of an item a in transaction Tid_1 , its utility is $3 \times 5 = 15$.

Any of the HUIM algorithms can also be used to find frequent itemsets if all the external utilities are set to 1 (Fournier-Viger et al., 2019).

Since HUIM is generalization of the FIM task, it shares the some of the difficulties – it is also very computationally challenging. The number of generated high utility itemsets tends to be very large (for an n distinct items there are $2^n - 1$ possible itemsets – the very same as for frequent itemsets). The difference comes with filtering the search space. Where FIM relies on anti-monotone property, the utility is neither monotone nor anti-monotone (Fournier-Viger et al., 2019). That means, that there is no rule that all of the subsets of high utility item set are also high utility and otherwise for low utility itemsets, thus the search space is considerably larger for HUIM tasks. Therefore, even though HUIM algorithms can be used to find frequent itemsets, FIM algorithms cannot be directly used to find high utility itemsets.

3.1 High utility itemset mining algorithms

Several high utility itemset mining algorithms has been proposed. The differences between the algorithms are whether they use a depth-first or breadth-first search, the type of database representation that they use internally or externally, how they determine the next itemsets to be explored and how they compute the utility of itemsets to determine if they satisfy the condition to be considered a high utility itemset. High utility itemset mining algorithms are generally inspired by frequent itemset mining algorithms with some advancements to cope with the fact that the utility measure is neither monotone nor anti-monotone (Fournier-Viger et al., 2019).

In 2006 Yao & Hamilton proposed two algorithms based on pruning strategies using utility values of $(k - 1)$ -itemsets to predict the utility upper bound of a k -itemset and using these values to constrain the utility values of the k -itemsets at level k . A k -itemset can be pruned immediately (without calculating its actual utility value), if its utility upper bound is lower

than *minutil* threshold. (Yao & Hamilton, 2006). First of those two algorithms is called UMining, an Apriori-like algorithm. This algorithm is complete (it is guaranteed to discover all the high utility itemsets), however, the running cost of the algorithm can be quite high.

“The worst case occurs when all itemsets are high utility itemsets, and thus all combinations of the items are tested. The time required to compute the utility value for an itemset is $(n \times k \times C(f))$, where $C(f)$ is the complexity of utility function f . Since the number of combinations of all items is $C_1^m + C_2^m + \dots + C_m^m = 2^m$, the worst case time complexity is $O(n \times k \times C(f) \times 2^m)$.” (Yao & Hamilton, 2006). In these formulas, n stands for number of transactions and m stands for total number of distinct items.

The second algorithm, called UMining_H, applies heuristic approach to search the search space. This algorithm does not guarantee to find all high utility itemsets, thus is incomplete. It combines utility upper bound property with Apriori property to create expected utility upper bound. The reason, why it may prune some of the high utility itemsets, is that when it calculates expected utility upper bound, it takes in account at most the minimum support of any itemset of size $(k - 1)$. However, when the size of the itemset increases, the support may be decreased due to fewer transactions being included (Yao & Hamilton, 2006). Based on the pruning strategies used for these algorithms, the actual utility of an itemset, which utility upper bound is lower than *minutil* is not calculated at all. As authors state and demonstrate in the related paper, it is possible, that the expected utility upper bound is lower than *minutil* even though the actual utility is greater than *minutil*. That way, some of the high utility itemsets can be missed. With a reasonable balance between accuracy and efficiency, the unwanted pruning of high utility itemsets can be avoided.

Most of the early algorithms (the problem of high utility itemset mining as its mostly known today was defined back in 2005, however the term has been used before) shares the same problem of being incomplete, mainly due to the use of heuristic strategies to reduce the search space (Fournier-Viger et al., 2019).

3.1.1 Two phase algorithms

Two phase algorithms are labeled by Fournier-Viger et al. (2019) as the first complete algorithms. As name suggests, two phase algorithms run in two phases:

1. calculation of transaction-weighted utilization (TWU) of itemsets and pruning all the supersets of itemsets where $TWU < minutil$, each itemset, that is not pruned is a high utility itemset candidate
2. calculation of exact utility of an itemset and if its utility is greater or equal to *minutil* then it is passed on as an output

As can be seen from the description of the phases, the two-phase algorithms use a measure, called transaction weighted utilization, to prune the search space. The TWU was introduced in 2005 by Liu et al. along with an algorithm called Two-Phase and it is used to prune the search space.

Transaction-weighted utilization (TWU) of an itemset I is the sum of transaction utilities of all the transaction containing I (Liu et al., 2005; Fournier-Viger et al., 2019).

Transaction utility (TU) is the sum of all utilities of the items in the transaction (Liu et al., 2005; Fournier-Viger et al., 2019).

The paper by Liu et al. (2005) introduces theorem transaction-weighted downward closure property. This property indicates, that any superset of a low transaction-weighted utilization itemset is also low in transaction weighted-utilization. The idea is that it is possible to define a monotone measure that is upper bound on the utility measure. There is notable similarity to the Apriori property.

“Let there be an itemset X – the TWU of X is no less than its utility $TWU(X) \geq u(X)$, the TWU of X is no less than the utility of its supersets $TWU(X) \geq u(Y) \forall Y \supset X$, therefore $TWU(X)$ must be greater or equal to the utility of X and any of its supersets.” (Fournier-Viger et al., 2019).

Two phase process of high utility itemset mining ensures pruning only low utility itemsets from the search space thus finding all high utility itemsets while improving the performance of algorithms (Liu et al., 2005; Fournier-Viger et al., 2019).

Two-Phase algorithm was the breakthrough algorithm, that introduced transaction-weighted downward property to prune the search space. The Two-phase algorithm is a generalization of the Apriori algorithm. Same as Apriori, Two-phase uses breadth-first search to explore the search space (Fournier-Viger et al., 2019).

Quantitative transaction database and *minutil* threshold are the input – the algorithm then in the first phase generates 1-itemsets and calculates their TWU. In each iteration the algorithm attempts to find larger itemsets by combining pairs of $(k - 1)$ - itemset. If an itemset X has a low utility $(k - 1)$ -subset, then X cannot be a high utility itemset and can be pruned. In the second phase exact utility is calculated and all itemsets, that were identified as high utility itemsets, are returned as output (Fournier-Viger et al., 2019).

Fournier-Viger (2019) addresses some limitations of Two-Phase algorithm – the first one being candidate generation. The algorithm does not check the database, whether the candidate itemset even exists in database, thus can create patterns, that do not appear in database. This can hinder the performance, because the algorithm can spend significant amount of time processing such itemsets. Other limitations addressed are the repeated database scans and cost in terms of memory, because Two-Phase has to keep all the high utility itemset candidates in memory before the second phase. The second phase, where the exact utilities are calculated, is generally considered very costly (Fournier-Viger et al., 2019).

Pseudo-code

Code 3.1 Two-Phase algorithm pseudo-code (Fournier-Viger et al., 2019)

```
Scan the database to calculate  $TWU$  of all items in  $I$ ;  
 $P_1 = \{i | i \in I \wedge u(\{i\}) \geq minutil\}$ ;  
 $k = 2$   
while  $P_k \neq \emptyset$  do  
     $P_k = itemsetGeneration(P_{k-1})$ ;  
    Remove each candidate  $X \in P_k$  that contains a  $(k-1)$ -itemset that is not in  $P_{k-1}$ ;  
    Scan the database to calculate the  $TWU$  of each candidate  $X \in P_k$ ;  
     $P_k = \{X | X \in P_k \wedge TWU(X) \geq minutil\}$ ;  
     $k = k + 1$   
end while  
 $P = \cup_{k=1 \dots k} P_k$ ;  
Scan the database to calculate the utility of each itemset in  $P$ ;  
return each itemset  $X \in P$  such that  $u(X) \geq minutil$ ;
```

Apart from the Two-Phase algorithm other two-phase algorithms are for example IHUP, UP-Growth, HUP-Growth and MU-Growth (Fournier-Viger et al., 2019).

3.1.2 Pattern-Growth Two Phase Algorithms

Pattern-growth algorithms have been proposed to address some of the drawbacks of Two-phase algorithm. The concept is similar to the previously mentioned FIM algorithm FP-Growth. The main idea of pattern-growth algorithms is to scan database to find itemsets – this prevents creation of itemsets that do not appear in database. Pattern-growth algorithms, to reduce the cost of scanning the database, have introduced the concept of projected database to reduce the size of databases (Fournier-Viger et al., 2019).

“The projected database of an item i is defined as the set of transactions where i appears but where the item i and items preceding i according to $<$ order have been removed.” (Fournier-Viger et al., 2019)

The search starts from 1-itemset and to create larger itemsets other items are appended recursively. The projected database is used for depth-first search after the 1-itemsets high utility candidates are found and stored in the set Z . The current itemset X starts as an empty set, for each item in Z , the itemset $X \cup \{z\}$ is stored as candidate high utility itemset. Then through the depth-first search larger frequent itemsets that are extensions of $X \cup \{z\}$ are found. Most pattern-growth algorithms utilize a depth-first search to explore the search space. Depth-first search has the advantage that less itemsets need to be kept in memory during the search. Since not all items in database can be appended to $X \cup \{z\}$ (the itemset $X \cup \{z\}$ alone does not need to appear in all transactions), the projected database of $X \cup \{z\}$ is created and the search is done over the projected database (Fournier-Viger et al., 2019). The second phase, where exact utilities are calculated remains the same as for regular two-phase algorithms.

Among pattern-growth two phase algorithms belong for example IHUP, UP-Growth, HUP-Growth or MU-Growth (Fournier-Viger et al., 2019).

3.1.3 One phase algorithms

The two-phase process of mining high utility itemsets, as has been explained, can still be rather costly, due to the huge number of candidates generated in the first phase, similarly to the FIM algorithms described earlier.

After the breakthrough idea of Two-Phase and TWU measure, the second breakthrough in HUIM has been the one phase algorithms – algorithms, that do not generate candidate itemsets. The utility of an itemset is immediately calculated – and thus identified as high or low utility itemset – and there is no need to store candidates in memory (Fournier-Viger et al., 2019).

Popular types of one phase algorithms are those based on utility-list structure, such as for example HUI-Miner or FHM algorithm. The utility-list is created for each visited itemset, more shorter ones can be joined to create utility-lists for larger patterns, and it contains information:

- about the utilities of the itemset in the transactions where it appears
- about the utilities of remaining items in such transactions

(Fournier-Viger et al., 2019).

“Utility-list – let there be an itemset X , assume, that a total order $>$ is defined on the set of items I appearing in the database – the utility-list $ul(X)$ of X is a set of tuples $(tid, iutil, rutil)$ for each transaction T_{tid} containing X , the $iutil$ is the utility of X in T_{tid} , the $rutil$ is defined as $\sum_{i \in T_{tid} \wedge i > x \forall x \in X} u(i, T_{tid})$.” (Fournier-Viger et al., 2019)

The utility-list structure enables to directly obtain utility of an itemset without scanning the database – the sum of $iutil$ values is equal to the utility of that itemset (Fournier-Viger et al., 2019).

The utility-list is also used for pruning based on new property called remaining utility upper-bound (Fournier-Viger et al., 2019).

For an itemset X , the remaining utility upper-bound (reu) is the sum of $iutil$ and $rutil$ values in its utility-list, the relationship $u(Y) \leq reu(X)$ is true for any itemset Y that is an extension of X (Fournier-Viger et al., 2019).

Based on this property, if itemset X has its utility lower than $minutil$, X and all its extension can be pruned from the search space, since they cannot be high utility itemsets.

Fournier-Viger (2019) lists some drawbacks of the utility-list based algorithms – first, it may explore itemsets, that do not exist in database, since it can create utility-lists by joining them without looking at the database. Second, the utility-list is built for every visited itemset (in the worst case $O(n)$ where n is the number of transactions (Zida et al., 2015)), which can be costly in the memory department.

3.1.4 Patter-Growth One-Phase Algorithms

Pattern-growth one-phase algorithms explore the search space by reading the database – this address one of the limitations of the utility-list based one phase algorithms – they only consider patterns that exists in the database.

One of the best-known algorithms in this group, EFIM algorithm (extension of FIM LCM algorithm), proposed in 2015 by Zida et al. introduced two novel-upper bounds called the local-utility and subtree-utility and novel array-based utility counting technique called Fast Utility Counting to calculate these upper-bounds in linear time and space. The EFIM algorithm utilizes a depth-first search and its search space of can be represented as a set-enumeration tree – there is presumed order on items (EFIM defines the order as the order of increasing TWU). The two novel-upper bounds are defined in concern to the set-enumeration tree structure. It also uses projected database and reduces the cost of database scans by transaction merging – identical transactions are replaced by single transaction (Zida et al., 2015).

Zida et al. (2015) in the related paper compare EFIM's performance (execution time, memory usage and number of visited nodes) on various datasets to other one phase algorithms – FHM algorithm, HUI-Miner, HUP-Miner, D2HUP and UP-Growth algorithm. The results show that EFIM is in general two to three orders of magnitudes faster and consumes up to eight times less memory.

4 High utility action rules mining

Up to this point, three data mining methods have been presented – FIM, action rules mining and HUIM. The overall goal of data mining is to derive useful information from data and each of the three methods previously mentioned provides user different kind of information. The concept of high utility action rules takes the goals of action rules mining and HUIM and combines them to form the goal of the high utility action rules mining – to provide user information about the most profitable actions.

Su et al. in 2012 proposed a MVAR algorithm for mining action rules with higher expected utilities. Expected utility of a rule r is defined as the sum of averages of utilities of action (change in flexible attributes) and effect (change in decision attribute) multiplied by the confidence of the rule. The expected utility was proposed to conquer some restrictions to the cost measures (negative utility), such as that the cost measures do not consider the benefit brought by a change in the decision attribute (Su et al., 2012). However, the concept of expected utility is not considered in the proposed implementation mainly, because the process of action rules generation used by MVAR is different to the one used in ActionRules package. The ActionRules package uses loosely coupled framework for action rules generation whereas MVAR generates them from the frequent action sets (similar principle to regular frequent itemsets).

The task of high utility action rules mining has been implemented as an extension of an ActionRules package and its features are discussed further.

4.1 Package ActionRules

ActionRules package is an implementation for action rules discovery available for Python. First version was released by Sýkora & Kliegr in 2020.

The package implements a loosely coupled framework for action rules discovery – the complete mining workflow uses Apriori-C algorithm for mining classification rules, that are used for action rules creation. However, the package also can be used for already existing classification rules.

The classification rules are stored in the decision table, which is split in two – the table with the desired target class (value of decision attribute d) and the table without desired target class. The action rules are generated from pairs of classification rules – these pairs are created as a Cartesian product of the rows in these two tables. The baseline approach, to check whether a pair of classification rules can be used for generation of an action rule checks, if the same stable attributes are present in both rules and if their values are the same. Similarly, the same flexible attributes must be present in both rules, but their values must be different (Sýkora & Kliegr, 2020).

The decision table can be split into a number of smaller tables – this step allows pruning of the search space and overall, increasing the speed of the action rule mining process. The tables are generated for each stable attribute, as they always remain unchanged (Sýkora & Kliegr, 2020). This step is optional.

Once all candidate pairs are processed, the result action rules are printed as an output. The package provides several output options for action rule representation.

The workflow of the ActionRules package is depicted in the flow diagram in section 4.2.1.

4.2 Extension for high utility action rules mining

The added functionality is demonstrated on Employee Attrition dataset from Kaggle website, provided by Sonali Shanbhag (available from [Employee Attrition | Kaggle](#)). This dataset is designed for classification, it aims to predict whether the employee of an unnamed company will leave or not based on various criteria, such as their salary. It contains information about 100 employees in total of seven defining attributes and binary target attribute. The only pre-processing of the dataset consisted of renaming the *BusinessTravel* attribute as the original dataset has a space between the two words. The Jupyter notebook (Annex A) used for demonstrating and testing of the added functionalities is enclosed with this paper along with the source dataset (Annex B) and table with utility values (Annex C).

	Gender	Satisfaction	BusinessTravel	Department	EducationField	Salary	Home-Office	Attrition
0	Female	High	Rare	Sales	Engineering	Medium	Near	Yes
1	Male	Low	Frequent	Sales	Engineering	Low	Near	Yes
2	Male	Medium	Rare	R&D	Other	Medium	Far	No
3	Female	Low	Frequent	R&D	Engineering	Medium	Far	Yes
4	Male	Medium	Rare	R&D	Medical	Medium	Far	No

Figure 4.1 Head of Employee Attrition dataset (output Jupyter Notebook)

The extension for high utility action rules mining is used, when the named parameters (*utility_source*, *min_util_dif*) are specified, when calling one of the *fit()* methods.

```
actionRulesDiscovery.fit(stable_attributes=["Department", "EducationField", "Satisfaction"],
                        flexible_attributes=["Salary"],
                        consequent="Attrition",
                        conf=40,
                        supp=3,
                        desired_classes=["No"],
                        utility_source=utility_source,
                        min_util_dif=100,
                        )
```

Figure 4.2 Input parameters for high utility action rules mining (output Jupyter Notebook)

The exact same input parameters as shown in figure above were used for high utility action rules mining for Employee Attrition dataset (results shown in the latter sections of this chapter).

4.2.1 Flow diagram of the extended ActionRules package

Following graphic depicts the workflow of the extended ActionRules package for high utility action rules mining. The diagram follows ISO 5807 notation.

The blue processes represent the original ActionRules workflow. The green processes are the extension, that enables the high utility action rules generation.

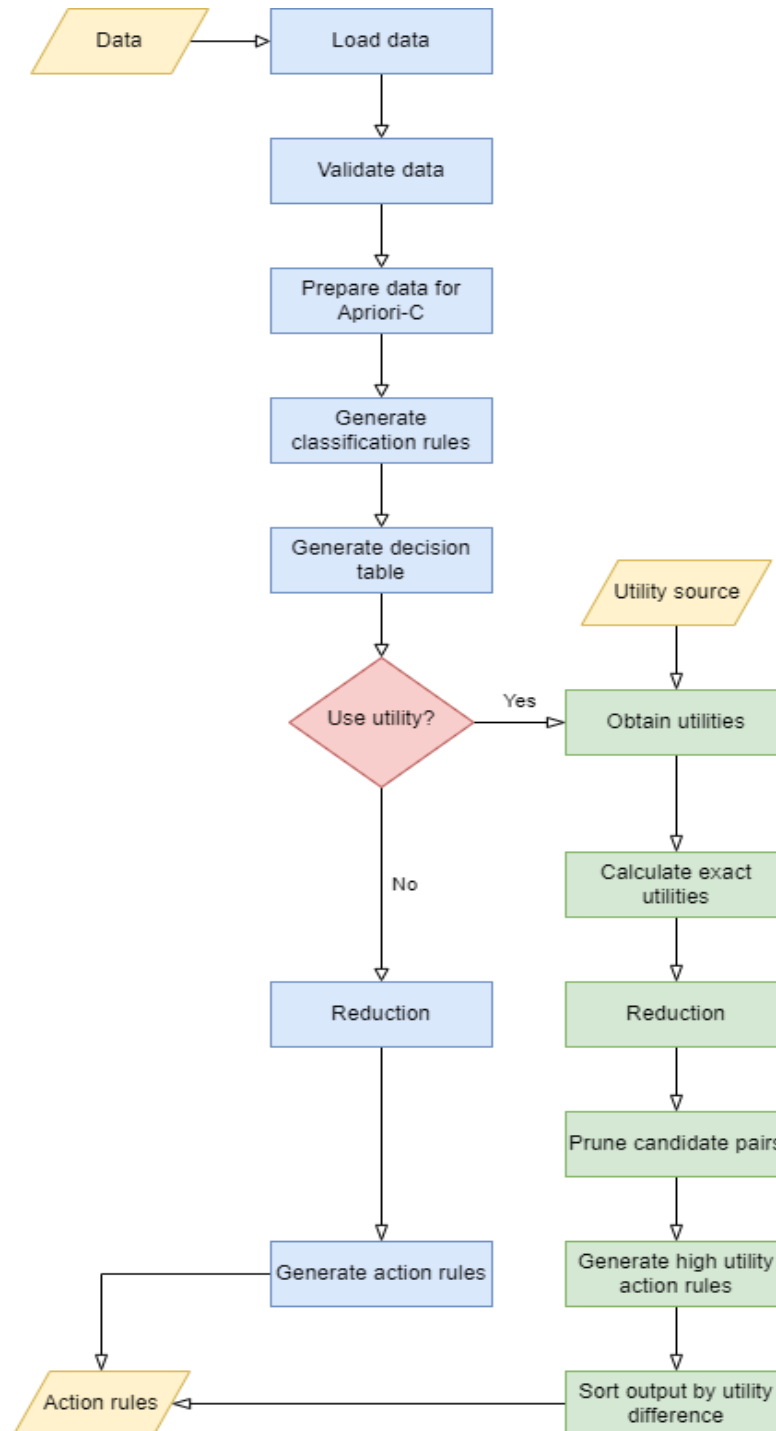


Figure 4.3 Flow diagram of extended ActionRules package

4.2.2 Utility source

Every attribute value has assigned its utility value. These values are defined by user and must be a positive number or zero. The utility source is not strongly typed – the extended package currently supports utility source in form of DataFrame (utility table) or function (utility function).

Utility table is a two-dimensional DataFrame, which index must be in form of *attribute_attributeValue*. The second column contains the utility value as float. The *attributeValue* part of the index must always be written in the lower case, since the package code during the process of discovering action rules converts all these values to lower case.

```
utility_source=pd.read_csv("attrition_utility.csv", sep=";",header=None, index_col=0)
```

Figure 4.4 Loading utility table for Employee Attrition dataset (output Jupyter Notebook)

User has a freedom in term of creating the utility table – the utility table can be created independently. The simplest way is to use *pandas* functions for handling CSV file.

BusinessTravel_no	400
BusinessTravel_rare	200
BusinessTravel_frequent	100
Department_sales	10000
Department_r&d	5000
EducationField_engineering	500
EducationField_other	500
EducationField_medical	500
Salary_low	500
Salary_medium	200
Salary_high	100
Attrition_yes	0
Attrition_no	1000

Figure 4.5 Utility table for Employee Attrition dataset (output Jupyter Notebook)

It is important to mention, that utilities in the utility table shown in figure 4.5 are just illustrational and are supposed to reflect, which attribute values are more profitable from the company point of view. For example, paying lower salary is more profitable to the company and frequent business trips can be quite costly.

The extension contains method `calculate_utilities()`, which for every classification rule in the input list calculates its utility. The utility of classification rule is a sum of utilities of all flexible and target attribute values present in the rule. The reason stable attributes are omitted, is because action rules in general aims to discover and suggest change in flexible

attributes that will result in change of the target. Therefore, the main focus in high utility action rules mining is on the utility of the suggested change. While suggesting the change, only flexible and target attributes are considered – the stable attributes, by definition, remains the same, thus there is no difference in utility caused by action.

Method `calculate_utilities()` takes as input all flexible and target attributes values present in the classification rules. Based on the attribute and attribute values, the method `_get_utility()` calculates utilities of classification rules. The `_get_utility()` method expects input set of parameters in form of *attributeName = attributeValue*. For passing these parameters to the `_get_utility()` method the dictionary structure is used.

Code 4.1 `_get_utility` function

```
def _get_utility(self, **kwargs):
    if callable(self.utility_function):
        utility = 0
        for key, value in kwargs.items():
            param = {}
            param[key] = value
            utility += self._check_utility(self.utility_function(**param))
        return utility
    if isinstance(self.utility_table, pd.DataFrame):
        utility = 0
        for key, value in kwargs.items():
            index = key + '_' + value
            try:
                utility += self._check_utility(self.utility_table.at[index, 1])
            except KeyError:
                print('Warning - key error at index ', index)
        return utility
    return 0
```

Method `_check_utility()` checks the validity of utility value. Currently, the method checks whether the value is nonnegative – when negative, 0 is used instead and warning is printed out. If the utility values for corresponding attributes are not found in the utility source, they are set as 0 and warning is printed as well.

The utility function should always return one utility value for each attribute and attribute value. The only restriction in the utility function structure is its input parameters and its output – the processing of the utility values for each attribute value is entirely up to user.

```
def utility_function(**kwargs):
    if len(kwargs.items()) == 0:
        return 0
    name, value = list(kwargs.items())[0]
    if name == 'BusinessTravel':
        if value == 'no':
            return 400
        if value == 'rare':
            return 200
        if value == 'frequent':
            return 100
    if name == 'Department':
        if value == 'sales':
            return 10000
        if value == 'r&d':
            return 5000
    # etc
    if name == 'Attrition':
        if value == 'yes':
            return 0
        if value == 'no':
            return 1000
    return 0
```

Figure 4.6 Utility function example with values for Employee Attrition Dataset (output Jupyter Notebook)

4.2.3 Pruning candidate pairs by difference in utility

After calculating utilities each classification rule is extended by its utility. Once candidate pairs are generated, the utility difference (utility of the action) is calculated by subtracting utility of the rule_after and rule_before. As has been mentioned, one of the input parameters is min_util_dif. This parameter is user defined threshold for difference in utility caused by action, thus if difference is lower than this threshold the processing of such candidate pair is skipped.

The result difference can be, unlike the input utilities, negative. These actions, to provide correct context, can be further estimated by domain experts (Su et al., 2012).

Code 4.2 Utility difference check

```
util_dif = None
if util is not None:
    utility_before = util[rule_before_index]
    utility_after = util[rule_after_index]
    util_dif = utility_after - utility_before
    if self.min_util_dif > util_dif:
        continue
```


The output action rules, in addition to current properties, are generated with the utility difference value.

```
["If attribute 'Department' is 'sales', attribute 'Salary' value 'low' is changed to 'high', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.04, confidence: 0.6666666666666666 and uplift: 0.04. Change in utility caused by action is 600.0.",
 "If attribute 'Department' is 'sales', attribute 'Salary' value 'low' is changed to 'medium', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.04, confidence: 0.48484848484848486 and uplift: 0.023636363636363636. Change in utility caused by action is 700.0.",
 "If attribute 'Satisfaction' is 'low', attribute 'Salary' value 'low' is changed to 'high', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.03, confidence: 0.6428571428571428 and uplift: 0.024285714285714285. Change in utility caused by action is 600.0.",
 "If attribute 'Satisfaction' is 'low', attribute 'Salary' value 'low' is changed to 'medium', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.03, confidence: 0.6000000000000001 and uplift: 0.022. Change in utility caused by action is 700.0."]
```

Figure 4.7 Output of high utility action rules (output Jupyter Notebook)

4.2.4 Sorting output action rules by utility of action

One of the optional input parameters is `sort_by_util_dif`. This parameter is by default set to *False*. If set to *True*, the output action rules are sorted descending by utility difference.

```
if util is not None and self.sort_by_util_dif:
    self.action_rules.sort(key=lambda rule: rule[4], reverse=True)
```

The index [4] is the position of the added utility difference in the action rule.

```
["If attribute 'Department' is 'sales', attribute 'Salary' value 'low' is changed to 'medium', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.04, confidence: 0.48484848484848486 and uplift: 0.023636363636363636. Change in utility caused by action is 700.0.",
 "If attribute 'Satisfaction' is 'low', attribute 'Salary' value 'low' is changed to 'medium', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.03, confidence: 0.6000000000000001 and uplift: 0.022. Change in utility caused by action is 700.0.",
 "If attribute 'Department' is 'sales', attribute 'Salary' value 'low' is changed to 'high', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.04, confidence: 0.6666666666666666 and uplift: 0.04. Change in utility caused by action is 600.0.",
 "If attribute 'Satisfaction' is 'low', attribute 'Salary' value 'low' is changed to 'high', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.03, confidence: 0.6428571428571428 and uplift: 0.024285714285714285. Change in utility caused by action is 600.0."]
```

Figure 4.8 Sorted output of high utility action rules (output Jupyter Notebook)

4.2.5 Output rules interpretation

From the Employee Attrition dataset, the aim is to get an information about what kind of employee would be convinced to stay by a salary raise. For stable attributes *Department*, *EducationField* and *Satisfaction* attributes were chosen. Flexible attribute is *Salary*. Minimal difference in utility was set to 100 (other input parameter values can be seen in figure 4.5).

Two most profitable action rules (out of 4 mined in total) are:

```
"If attribute 'Department' is 'sales', attribute 'Salary' value 'low' is changed to 'medium', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.04, confidence: 0.48484848484848486 and uplift: 0.023636363636363636. Change in utility caused by action is 700.0."
```

"If attribute 'Satisfaction' is 'low', attribute 'Salary' value 'low' is changed to 'medium', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.03, confidence: 0.6000000000000001 and uplift: 0.022. Change in utility caused by action is 700.0."

When they are compared to the other two rules:

"If attribute 'Department' is 'sales', attribute 'Salary' value 'low' is changed to 'high', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.04, confidence: 0.6666666666666666 and uplift: 0.04. Change in utility caused by action is 600.0."

"If attribute 'Satisfaction' is 'low', attribute 'Salary' value 'low' is changed to 'high', then 'Attrition' value 'Yes' is changed to 'No' with support: 0.03, confidence: 0.6428571428571428 and uplift: 0.024285714285714285. Change in utility caused by action is 600.0."

it can be observed that employees, that would be convinced by a salary raise are the ones in the sales department and these that rated their satisfaction with the working environment in the company as low. The interesting observation here is fact, that company does not need to raise these salaries on a *high* level, because *medium* is not only more profitable but sufficient enough to convince certain number of these employees to stay.

It is important, while studying the results, to consider the input data – the chosen dataset is rather small as the focus was on choosing existing dataset that would be the best to model meaningful situation, where high utility action rules mining would be useful as well as easy enough to be understood by regular users rather than domain experts.

4.3 Alternative implementations

To the author's best knowledge, no other implementation for high utility action rules mining has been proposed to date. The SPMF Open-Source Data Mining Library Version 2 proposed by Fournier-Viger et al. in 2016 ([SPMF: A Java Open-Source Data Mining Library \(philippe-fournier-viger.com\)](http://philippe-fournier-viger.com)) offers Java framework for high utility pattern mining. Sýkora & Kliegr in the paper related to ActionRules package (2020) mention LISp-Miner developed at the University of Economics as a publicly available alternative for action rules mining to the ActionRules package.

4.4 Future development

The basic concept of high utility action rules mining has been successfully implemented – however, the framework can be improved and further developed. During the final code review and testing, several ideas for possible improvements and enhancements have been discovered.

Currently, the extension for high utility action rules mining has two options for loading utility sources – possible improvement in this area could consist of passing a file of certain type (XML, JSON, CSV etc.) and processing that file inside the module to build the DataFrame.

Another option for future development would be offering a unified way to check, whether the utility mining functionalities should be used instead of complex IF condition which is used at the moment. This improvement would present a new method that will cover the logic behind the decision whether the extension for high utility action rules mining should be used.

The extension added the possibility to sort the output based on utility difference – this functionality could be generalized to allow to sort by other criteria, for example by confidence ascending.

Future development could also add the concept of cost and profit. Currently, the utility values are only one dimensional – it is possible to interpret values as more or less profitable, but it is not really possible to consider contrasting measures.

Current implementation uses utilities as absolute values. The utility difference in the output rules does not reflect how the rule covers the dataset – the action alone could be profitable but might only cover small subset of data. In such situation a different action, that would in current implementation deem less profitable, could possibly bring higher profit when performed because it would have covered bigger portion of the data. To reflect how the action rule covers the dataset, confidence or support measure of the action rule may be used. This possible development requires further research.

Conclusions

All aims that has been set for this paper has been met. The first three chapters provide background information about key topics related to high utility action rules mining (FIM, action rules mining, HUIM) and ActionRules package. Over thirty scientific sources, primarily articles, have been studied to retrieve information contained in these chapters. Chapters provide not only insight on the goals and key terms of the concerned data mining method, but also provides examples of existing algorithms to give reader and idea about in which direction the development of retrieving information in that area goes and how it developed over time.

Practical solution for high utility action rules mining has been successfully implemented to the existing ActionRules package framework. The solution required different approach then discussed HUIM algorithms, since the input and overall characteristics of action rules is different from regular itemsets. Therefore, the extension was implemented with no existing HUIM algorithm in mind, rather than the focus was on understanding the importance of the suggested action and to create a solution that would reflect it.

Through debugging of a .py file in the PyCharm idea the dataflow of the original ActionRules package was observed to gain better understanding of the overall package workflow. These insights were also used to determine the correct place where the extension should be connected to already existing functionalities. This process of debugging served as a great experience on how to work with and understand already existing framework. After the implementation of the high utility action rules mining extension its functionalities have been tested. Tests consisted primarily of testing the utility values – for example setting negative values or not setting them at all. Other than that, situation when not all required input parameters are set have also been tested. All tests passed and program did not exhibit any unexpected or unwanted behaviour.

High utility action rules are way of acquiring additional, possibly more targeted, information from data, than regular action rules. There is, if any, limited number of frameworks for high utility action rules mining – there is potential in this field of knowledge discovery and should be definitely explored further.

List of references

- AGGARWAL, Charu C., 2014. Applications of Frequent Pattern Mining. In: AGGARWAL, Charu C. and HAN, Jiawei (eds.), *Frequent Pattern Mining*. Online. Cham: Springer International Publishing. pp. 443–467. [Accessed 28 March 2022]. ISBN 978-3-319-07820-5.
- AGGARWAL, Charu C., BHUIYAN, Mansurul A. and HASAN, Mohammad Al, 2014. Frequent Pattern Mining Algorithms: A Survey. In: AGGARWAL, Charu C. and HAN, Jiawei (eds.), *Frequent Pattern Mining*. Online. Cham: Springer International Publishing. pp. 19–64. [Accessed 28 March 2022]. ISBN 978-3-319-07820-5.
- AGRAWAL, Rakesh and SRIKANT, Ramakrishnan, 1994. Fast Algorithms for Mining Association Rules. In: *VLDB '94: Proceedings of the 20th International Conference on Very Large Data Bases*. San Francisco, CA, United States: Morgan Kaufmann Publishers Inc. 340 Pine Street, Sixth Floor, pp. 487–499.
- BORGELT, Christian, 2012. Frequent itemset mining: Frequent itemset mining. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*. November 2012. Vol. 2, no. 6, pp. 437–456. DOI 10.1002/widm.1074.
- CHMIELEWSKI, Michal R, GRZYMALA-BUSSE, Jerzy W, PETERSON, Neil W and THAN, Soe, 1993. The Rule Induction System LERS—A Version for Personal Computers. . 1993. pp. 32.
- COENEN, F. and LENG, P., 2004. An Evaluation of Approaches to Classification Rule Selection. In: *Fourth IEEE International Conference on Data Mining (ICDM'04)*. Online. Brighton, UK: IEEE. 2004. pp. 359–362. [Accessed 28 March 2022]. ISBN 978-0-7695-2142-8. DOI 10.1109/ICDM.2004.10012.
- DARDZINSKA, Agnieszka, 2013. *Action Rules Mining*. Online. Berlin, Heidelberg: Springer Berlin Heidelberg. [Accessed 28 March 2022]. Studies in Computational Intelligence. ISBN 978-3-642-35649-0.
- DONG, Guozhu and LI, Jinyan, no date. Efficient Mining of Emerging Patterns: Discovering Trends and Differences. . pp. 10.
- DONG, Guozhu, ZHANG, Xiuzhen, WONG, Limsoon and LI, Jinyan, 1999. CAEP: Classification by Aggregating Emerging Patterns. In: ARIKAWA, Setsuo and FURUKAWA, Koichi (eds.), *Discovery Science*. Online. Berlin, Heidelberg: Springer Berlin Heidelberg. pp. 30–42. Lecture Notes in Computer Science. [Accessed 31 March 2022]. ISBN 978-3-540-66713-1.

FOURNIER-VIGER, Philippe, LIN, Jerry Chun-Wei, NKAMBOU, Roger, TSENG, Vincent S. and VO, Bay (eds.), 2019. *High-Utility Pattern Mining: Theory, Algorithms and Applications*. 1st ed. 2019. Cham: Springer International Publishing : Imprint: Springer. Studies in Big Data, 51. ISBN 978-3-030-04921-8. 006.3

HAN, Jiawei, PEI, Jian and YIN, Yiwen, 2000. Mining Frequent Patterns without Candidate Generation. *ACM SIGMOD Record*. June 2000. Vol. 29, pp. 1–12. DOI 10.1145/335191.335372.

HE, Z., XU, X., DENG, S. and MA, R., 2005. Mining action rules from scratch. *Expert Systems with Applications*. October 2005. Vol. 29, no. 3, pp. 691–699. DOI 10.1016/j.eswa.2005.04.031.

IBM. Confidence in an association rule. In: *IBM Db2 documentation* [online]. 2021-03-01 [Accessed 30 March 2022]. Available from: https://www.ibm.com/docs/en/db2/10.5?topic=SSEPGG_10.5.0/com.ibm.im.model.doc/c_confidence_in_an_association_rule.html

JOVANOSKI, Viktor and LAVRAČ, Nada, 2001. Classification Rule Learning with APRIORI-C. *EPIA 2001: Progress in Artificial Intelligence*. 2001. pp. 44–51. DOI 10.1007/3-540-45329-6_8.

LIU, Bing, HSU, Wynne and MA, Yiming, 1998. Integrating Classification and Association Rule Mining. . pp. 7.

LIU, Ying, LIAO, Wei-keng and CHOUDHARY, Alok, 2005. A fast high utility itemsets mining algorithm. In: *Proceedings of the 1st international workshop on Utility-based data mining - UBDM '05*. Online. Chicago, Illinois: ACM Press. 2005. pp. 90–99. [Accessed 10 April 2022]. ISBN 978-1-59593-208-2. DOI 10.1145/1089827.1089839.

LUNA, José María, FOURNIER-VIGER, Philippe and VENTURA, Sebastián, 2019. Frequent itemset mining: A 25 years review. *WIREs Data Mining and Knowledge Discovery*. Online. November 2019. Vol. 9, no. 6. [Accessed 28 March 2022]. DOI 10.1002/widm.1329.

MADEIRA, S.C. and OLIVEIRA, A.L., 2004. Biclustering algorithms for biological data analysis: a survey. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*. January 2004. Vol. 1, no. 1, pp. 24–45. DOI 10.1109/TCBB.2004.2.

PAWLAK, Z., 1985. Rough sets and decision tables. In: SKOWRON, Andrzej (ed.), *Computation Theory*. Online. Berlin, Heidelberg: Springer Berlin Heidelberg. pp. 187–196. Lecture Notes in Computer Science. [Accessed 2 April 2022]. ISBN 978-3-540-16066-3.

RAŚ, Zbigniew W. and DARDZIŃSKA, Agnieszka, 2008. Action Rules Discovery without Pre-existing Classification Rules. In: CHAN, Chien-Chung, GRZYMALA-BUSSE, Jerzy W. and ZIARKO, Wojciech P. (eds.), *Rough Sets and Current Trends in Computing*. Online. Berlin, Heidelberg: Springer Berlin Heidelberg. pp. 181–190. Lecture Notes in Computer Science. [Accessed 3 April 2022]. ISBN 978-3-540-88423-1.

RAS, Zbigniew W. and WIECZORKOWSKA, Alicja, 2000. Action-Rules: How to Increase Profit of a Company. In: ZIGHED, Djamel A., KOMOROWSKI, Jan and ŻYTKOW, Jan (eds.), *Principles of Data Mining and Knowledge Discovery*. Online. Berlin, Heidelberg: Springer Berlin Heidelberg. pp. 587–592. Lecture Notes in Computer Science. [Accessed 28 March 2022]. ISBN 978-3-540-41066-9.

RAŚ, Zbigniew W., WYRZYKOWSKA, Elżbieta and WASYLUK, Hanna, 2008. ARAS: Action Rules Discovery Based on Agglomerative Strategy. In: RAŚ, Zbigniew W., TSUMOTO, Shusaku and ZIGHED, Djamel (eds.), *Mining Complex Data*. Online. Berlin, Heidelberg: Springer Berlin Heidelberg. pp. 196–208. Lecture Notes in Computer Science. [Accessed 3 April 2022]. ISBN 978-3-540-68415-2.

RAUCH, Jan and ŠIMŮNEK, Milan, 2009. Action Rules and the GUHA Method: Preliminary Considerations and Results. In: RAUCH, Jan, RAŚ, Zbigniew W., BERKA, Petr and ELOMAA, Tapio (eds.), *Foundations of Intelligent Systems*. Online. Berlin, Heidelberg: Springer Berlin Heidelberg. pp. 76–87. Lecture Notes in Computer Science. [Accessed 4 April 2022]. ISBN 978-3-642-04124-2.

SIKORA, Marek, MATYSZOK, Paweł and WRÓBEL, Łukasz, 2021. SCARI: Separate and Conquer Algorithm for Action Rules and Recommendations Induction. arXiv:2106.05348 [cs]. Online. 9 June 2021. [Accessed 2 April 2022]. Retrieved from: <http://arxiv.org/abs/2106.05348> arXiv: 2106.05348

SRIKANT, Ramakrishnan and AGRAWAL, Rakesh, 1997. Mining generalized association rules. *Future Generation Computer Systems*. November 1997. Vol. 13, no. 2–3, pp. 161–180. DOI 10.1016/S0167-739X(97)00019-8.

SU, Peng, LI, Dunfeng and SU, Kun, 2012. An expected utility-based approach for mining action rules. In: *Proceedings of the ACM SIGKDD Workshop on Intelligence and Security Informatics - ISI-KDD '12*. Online. Beijing, China: ACM Press. 2012. pp. 1–4. [Accessed 28 April 2022]. ISBN 978-1-4503-1550-0. DOI 10.1145/2331791.2331800.

SÝKORA, Lukáš and KLIEGR, Tomáš, 2020. Action Rules: Counterfactual Explanations in Python. . 2020. pp. 13.

WANG, Ke, XU, Chu and LIU, Bing, 1999. Clustering transactions using large items. In: *Proceedings of the eighth international conference on Information and knowledge management - CIKM '99*. Online. Kansas City, Missouri, United States: ACM Press. 1999. pp. 483–490. [Accessed 28 April 2022]. ISBN 978-1-58113-146-8. DOI 10.1145/319950.320054.

WANG, Ke, ZHOU, Senqiang and HE, Yu, 2000. Growing decision trees on support-less association rules. In: *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining - KDD '00*. Online. Boston, Massachusetts, United States: ACM Press. 2000. pp. 265–269. [Accessed 31 March 2022]. ISBN 978-1-58113-233-5. DOI 10.1145/347090.347147.

WENMIN LI, JIAWEI HAN, and JIAN PEI, 2001. CMAR: accurate and efficient classification based on multiple class-association rules. In: *Proceedings 2001 IEEE International Conference on Data Mining*. Online. San Jose, CA, USA: IEEE Comput. Soc. 2001. pp. 369–376. [Accessed 31 March 2022]. ISBN 978-0-7695-1119-1. DOI 10.1109/ICDM.2001.989541.

YAO, Hong and HAMILTON, Howard J., 2006. Mining itemset utilities from transaction databases. *Data & Knowledge Engineering*. December 2006. Vol. 59, no. 3, pp. 603–626. DOI 10.1016/j.datak.2005.10.004.

ZAKI, M.J., 2000. Scalable algorithms for association mining. *IEEE Transactions on Knowledge and Data Engineering*. June 2000. Vol. 12, no. 3, pp. 372–390. DOI 10.1109/69.846291.

ZAKI, Mohammed J and GOUDA, Karam, 2003. Fast Vertical Mining Using Diffsets. *KDD '03: Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. August 2003. pp. 326–335. DOI 10.1145/956750.956788.

ZIDA, Souleymane, FOURNIER-VIGER, Philippe, LIN, Jerry Chun-Wei, WU, Cheng-Wei and TSENG, Vincent S., 2015. EFIM: A Highly Efficient Algorithm for High-Utility Itemset Mining. In: SIDOROV, Grigori and GALICIA-HARO, Sofia N. (eds.), *Advances in Artificial Intelligence and Soft Computing*. Online. Cham: Springer International Publishing. pp. 530–546. Lecture Notes in Computer Science. [Accessed 13 April 2022]. ISBN 978-3-319-27059-3.

Annexes

Enclosed annexes consist of files used for demonstration and testing of the implemented extension.

Annex A: Jupyter notebook Employees

A Jupyter notebook containing the entire process of high utility action rules mining from user's point of view.

Annex B: Source dataset Employee Attrition

A source dataset (CSV) used for demonstration and testing. Also available from [Employee Attrition | Kaggle](#).

Annex C: Utility table for Employee Attrition dataset

A utility table (CSV) for Employee Attrition dataset.